

Research Note — LLD Practice Platform

The learner problem

Low-Level Design practice has an unusual property: it is easy to *start* and hard to *evaluate*. A learner can design a Parking Lot, an Elevator, or a Vending Machine in an afternoon and still not know whether their class boundaries, coupling, and extensibility choices are actually good — because, unlike an algorithm problem, there is no single correct answer to check against. Two designs that look very different (e.g. an inheritance-heavy `Spot` hierarchy vs. a strategy-object `PricingPolicy`) can both be defensible, and a learner working alone has no reliable way to tell which trade-offs they made deliberately versus by accident.

Three consequences follow from this, and they shaped the product direction more than any single feature request in the brief:

1. **Self-assessment doesn't work.** Asking a learner to rate their own coupling or cohesion is asking them to already know what they're practicing to learn. Any evaluation signal has to come from outside the learner's own judgement — either a fixed rubric applied deterministically, or a reviewer (human or AI) reasoning against that rubric.
2. **Feedback needs to point at something.** “Your design has a coupling issue” is not actionable; “`ParkingLot` directly constructs `Ticket` objects, so a pricing change would require editing `ParkingLot`” is. The learner needs evidence tied to the specific words they wrote, not a free-floating verdict.
3. **A single attempt teaches less than a sequence of attempts.** The interesting signal for a *practice* platform (as opposed to a one-shot assessment tool) is not “how did this attempt score” but “is the same weakness showing up across different problems.” That only exists if the platform remembers scored dimensions across attempts and groups them — otherwise every attempt is an island.

Existing approaches (researched by category, not by specific product)

The candidate research pass for this assignment reviewed the general landscape of LLD/system-design interview-practice tooling, coding judges, and AI code-review products, rather than compiling a list of specific named platforms — several specific tool names turned up during research but could not be independently re-verified live within this session's time budget, and citing an unverified product name would be worse than citing none. What is verifiable, and consistent across the category as a whole:

- **Coding judges (LeetCode-style)** solve the “is this correct” question well because the correctness check is fully deterministic (test cases pass or fail). They have no answer to LLD's central problem:

there is no test suite that certifies a class boundary is well-chosen. This category is a poor structural fit for LLD specifically, which is exactly why the brief separates LLD practice out as its own product rather than treating it as “another judge.”

- **Generic AI code-review tools** (chat-with-your-diff style assistants) solve the opposite problem: they can reason about design quality, but ask an unconstrained question (“is this a good design?”) and get an unconstrained, inconsistent answer back — the exact anti-pattern the assignment’s guide calls out in §7. Without a fixed rubric and a scope boundary, the same submission can score differently run to run, and the model tends to recommend infrastructure (event buses, plugin layers) far beyond what a Parking Lot problem actually calls for.
- **Structured mock-interview platforms** (rubric-scored or reviewer-graded) get the rubric idea right but are usually built around a live human reviewer, which doesn’t scale to “practice repeatedly, alone, on your own schedule” — the loop this assignment actually asks for.

The key gap

None of these categories close the loop across attempts. Each treats an attempt as a terminal event: you submit, you get a verdict, the interaction ends. The gap is a product that (a) evaluates a single attempt against a fixed, evidence-grounded rubric rather than an unconstrained “is this good” question, and (b) remembers scored dimensions across a learner’s attempts so that a *recurring* weakness — not just this attempt’s weakness — is visible. That second piece is the part every category above skips, and it’s also the cheapest piece to build once dimension scores are already being stored per attempt: it’s grouping and arithmetic, not a new AI capability.

Product direction taken

The platform commits to a narrow, evidence-grounded loop: choose problem → work in a structured Markdown template pre-filled with six required sections → submit → get per-criterion feedback (score, evidence quote, concern, why it matters, suggestion) from two evaluators running in parallel (a real rule-based evaluator scoring the 3 dimensions that are genuinely checkable without judgement, and an AI evaluator scoring all 8 against a fixed rubric with named anchors and a per-problem scope boundary) → review → try again. History surfaces per-dimension deltas against the learner’s previous attempt on the same problem, and flags dimensions that stay weak across *different* problems — not because recurring-weakness detection is a differentiator by itself, but because it’s the one piece of the loop the research above shows nothing else in the category actually does. Full reasoning for every decision behind this direction — including the specific rubric shape, the citation-verification mechanism, and the two-evaluator failure policy — is recorded in `DECISIONS.md`.